

Funciones en Python

En este documento encontrarás una serie de actividades para practicar el uso de distintas funciones en Python. ¡Vamos a programar! 🦉💻

Para la mayoría de los ejercicios de ésta guía no se detallan de manera explícita todos los argumentos y valores de retorno de las funciones requeridas. Cuando no se detallan en el enunciado estos elementos, la definición de la función, quedará a cargo del programador.

1 Actividad 1: Densidad de objetos ⚖️

📌 Consigna:

Necesitamos calcular la densidad de dos objetos.

1. Crear una función `calcular_densidad(masa, volumen)` que retorna la densidad ($\text{masa} / \text{volumen}$).
2. Usar la función para calcular la densidad de un objeto con:
 - Masa = 10 kg y Volumen = 2 m³
 - Masa = 270 kg y Volumen = 33 m³
3. Mostrar ambos resultados por consola.
4. Sumar ambas densidades y mostrar el resultado con el texto:

La densidad total es: _____

2 Actividad 2: Ataque al dragón 🐉🎲

📌 Consigna:

Crear un minijuego para tirar un dado y quitarle puntos de vida a un dragón.

1. Importar `random`.
2. Crear una variable `vida_dragon = 60`.
3. Crear la función `tirarDado(lados)` que reciba como argumento la cantidad de lados y devuelva un número aleatorio entre 1 y la cantidad de lados.
4. Crear la función `atacarDragon()` que sume el resultado de tirar un dado de 20 lados y uno de 4 lados, y retorne ese valor.
5. Restar el daño a `vida_dragon`.
6. Mostrar los puntos de vida restantes.

3 Actividad 3: Distancia entre dos puntos

Consigna:

Crear una función que reciba 4 argumentos: `x1`, `y1`, `x2`, `y2` y devuelva la **distancia** entre dos puntos en el plano.

1. Importar `math`.
2. Crear una función `distancia(x1, y1, x2, y2)` que reciba como argumentos los puntos del plano y devuelva la distancia calculada. Utilizar la fórmula de distancia:

$$distancia = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. Usar la función `math.sqrt()` para calcular la raíz cuadrada.
4. Llamar a la función con los valores (1, 2) y (4, 6).
5. Mostrar el resultado por consola.

4 Actividad 4: Conversión de temperaturas

Consigna:

Desarrollar un programa que permita cargar y convertir una temperatura en grados Celsius (°C) (comprendida entre -500 y 15000) a su equivalente en grados Fahrenheit (°F). Deberá validarse que la temperatura ingresada se encuentre en el intervalo requerido antes de pasar a la conversión de la misma. Tener en cuenta que el diseño debe implementar las siguientes funciones:

- Función `validaCelsius()`: permite validar que la temperatura ingresada en °C se encuentre comprendida en el intervalo [-500, 15000] informando la situación con un mensaje en pantalla. Esta función recibe como argumento la temperatura ingresada y devuelve como valor de retorno una bandera que indique el resultado de la validación.
- Función `convierteCaF()` permite convertir la temperatura de °C a °F. Dicha función recibe como argumento la temperatura en °C y devuelve como valor de retorno la temperatura en °F.

Recordamos que la fórmula matemática para convertir °C a °F es la siguiente:

$$^{\circ}F = \frac{9}{5} * ^{\circ}C + 32$$

5 Actividad 5: Cubo de un número real

Consigna:

Desarrollar un programa que utilizando una función llamada cubo() permita calcular el cubo de un número real. Antes de invocarla para obtener el resultado, se debe validar que el número ingresado por el usuario sea de tipo real utilizando una función llamada validaReal(). Aclaremos que cuando el número ingresado no sea real se debe volver a solicitar y validar hasta que cumpla con lo requerido.

6 Actividad 6: Validación de contraseña

Consigna:

Desarrollar un programa que, por medio de funciones, permite validar la contraseña ingresada por un usuario (la cual debe ser igual a 'supersecreta_156'). Se deberán implementar las siguientes funciones en la resolución:

- Función valida_intento(): controla si el usuario aún tiene intentos disponibles. Recibe como argumento la cantidad de intentos realizados hasta el momento y la cantidad máxima de intentos permitida y devuelve como valor de retorno una variable de tipo bandera (True o False) que indica si aún le quedan intentos o no.
- Función valida_pass(): determina si la contraseña es correcta o no. Recibe como argumento la contraseña y devuelve como valor de retorno una variable de tipo bandera (True o False) que indica si la contraseña es correcta o no.

La cantidad máxima de intentos permitidos para la contraseña es de 3 (tres) y la comprobación realizada en cada intento debe informarse con un mensaje en pantalla tal como 'Contraseña correcta' o 'Contraseña incorrecta' según corresponda. Se sobreentiende que, si se ingresa la contraseña correcta, se debe informar y finalizar el programa sin completar los intentos faltantes.

7 Actividad 7: Compara números

Consigna:

Desarrollar un programa que permita cargar tres números enteros y por medio de funciones determine lo siguiente:

- Función menor(): determina el menor de los tres números ingresados. Recibe como argumento los tres números ingresados y devuelve el menor.
- Función mayor(): determina el mayor de los tres números ingresados. Recibe como argumento los tres números ingresados y devuelve el mayor.
- Función signo(): permite determinar si los tres números ingresados fueron todos del mismo signo, es decir, todos positivos o todos negativos, o si hay valores positivos y negativos mezclados en la serie ingresada. Recibe como argumento los 3 números ingresados y devuelve una bandera que indica uno de los dos estados posibles. -
- Función compartenSigno(): es invocada sólo cuando todos los números ingresados fueron todos del mismo signo y permite generar la suma de los números multiplicada por el mayor encontrado anteriormente

8 Actividad 8: Procesamos números en un menú de opciones

Consigna:

Desarrollar un programa controlado por un menú de opciones generado desde una función llamada menu(), invocada desde el algoritmo principal, y el cual cuenta con las siguientes opciones:

- Opción 1 – Adivina el número secreto: por medio de una función llamada adivina() generar un número secreto al azar comprendido en el intervalo [0, 50] y determinar si un número entero y positivo n ingresado por el usuario es igual al número secreto generado por la computadora. La función recibe como argumento el número n ingresado y devuelve como valor de retorno una bandera que indica si el número n es igual al número secreto. Antes de salir de la opción informar con un mensaje en pantalla si el usuario ganó o no ganó el juego.

- Opción 2 – Capicúas en el intervalo: dado un intervalo cerrado [desdehasta], ingresado por el usuario, determinar por medio de una función llamada esCapicua(), todos los números capicúas contenidos en el intervalo y mostrarlos. La función recibe como argumentos los extremos desde-hasta del intervalo ingresados por el usuario. Antes de llamar a la función se deberá validar que los extremos del intervalo sean positivos y que el extremo del intervalo desde no sea mayor que el extremo del intervalo hasta.
- Opción 3 – Salir.

Nota: puedes intentar combinarlo con el ejercicio de la actividad 6. En este caso solo deberías permitir el acceso al menú cuando se ingrese la contraseña correcta dentro de la cantidad permitida de intentos.

9 Actividad 9: Procesamos texto con menú de opciones

Consigna:

Desarrollar un programa que permita procesar una cadena de caracteres. El programa será controlado por un menú de opciones generado desde una función llamada menú(), invocada desde el algoritmo principal y la cual cuenta con las siguientes opciones:

- Opción 1 – Leer una cadena: permite ingresar una cadena de caracteres desde el teclado. Desarrollar una función llamada leeCadena() que no reciba ningún argumento y devuelva una cadena de caracteres leída desde el teclado.
- Opción 2 – Contar minúsculas: permite contar la cantidad de letras minúsculas de la cadena de caracteres ingresada con la opción 1. Para los conteos se deben incluir: letras consonantes minúsculas, vocales minúsculas sin acentuar y acentuadas con tilde ortográfico ‘’). La opción se debe implementar por medio de una función llamada cuentaMin().
- Opción 3 – Contar consonantes: permite contar la cantidad de letras consonantes de la cadena de caracteres ingresada con la opción 1. La opción se debe implementar por medio de una función llamada cuentaConsonantes().
- Opción 4 – Contar números: permite contar la cantidad de caracteres numéricos de la cadena de caracteres ingresada con la opción 1. El conteo debe incluir los caracteres numéricos del ‘0’ al ‘9’. La opción se

debe implementar por medio de una función llamada `cuentaNumeros()`.

- Opción 5 – Contar carácter: permite contar la cantidad de veces que aparece un carácter dentro de la cadena ingresada con la opción 1. El carácter a contar será ingresado por el usuario. La opción se debe implementar por medio de una función llamada `cuentaCaracter()`.
- Opción 6 - Salir.

10 Actividad 10: Juego de lotería

Consigna:

Simule un juego de lotería para 5 jugadores. Cada jugador ingresará el número para su jugada por medio de una función llamada `jugada()`, el número ingresado debe estar comprendido en el intervalo $[0, 100]$. Luego de ingresar todas las jugadas, el programa debe simular el juego a través de una función llamada `lotería()`. Ésta función genera al azar un número aleatorio, también comprendido en el intervalo $[0, 100]$, y determina si hubo algún ganador entre los 5 jugadores (o más de uno) informado la situación con un mensaje en pantalla.].

Nota: en Python, podrá definir las variables correspondientes como globales o aprovechar la característica propia de Python que permite devolver más de un valor de retorno.